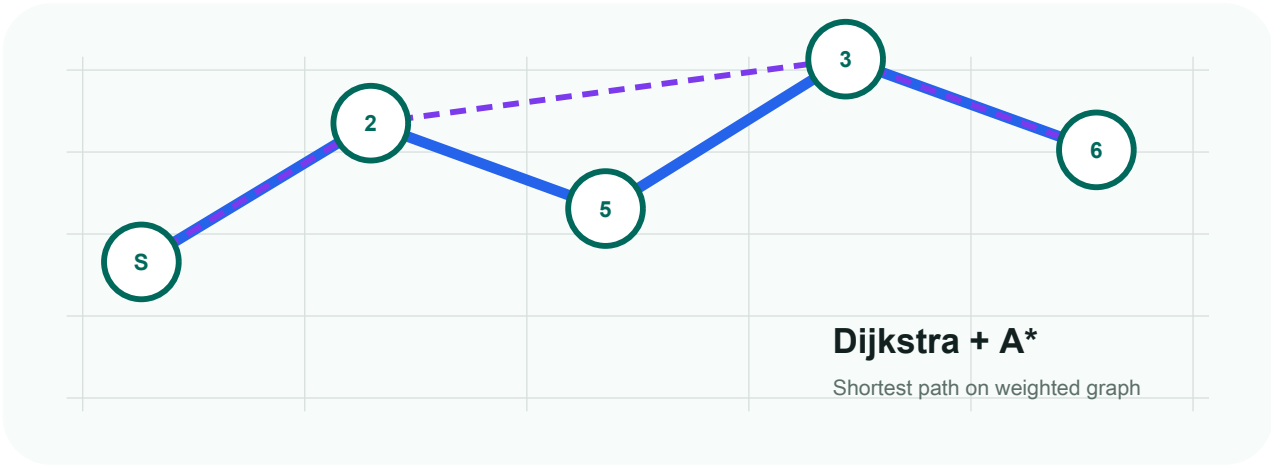
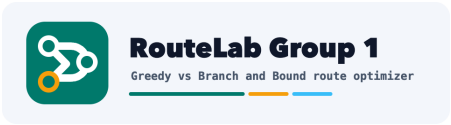




BÁO CÁO DỰ ÁN

RouteLab Group 1

Mô phỏng tìm đường ngắn nhất trên bản đồ bằng Dijkstra và A*. Ứng dụng dùng graph có trọng số, chọn một điểm nguồn và một điểm đích, sau đó so sánh path, tổng chi phí, node đã duyệt và runtime.

Hạng mục	Nội dung
Môn học	Phân tích thiết kế giải thuật
Tech stack	React + Vite · Node.js + Express · Leaflet/OpenStreetMap · JSON graph
Demo scope	Graph 5-10 điểm, trọng số không âm, tìm đường từ source đến target
Dataset chính	Ho Chi Minh City shortest-path graph (7 nodes, 12 edges)
Domain frontend	maps.hailamdev.space

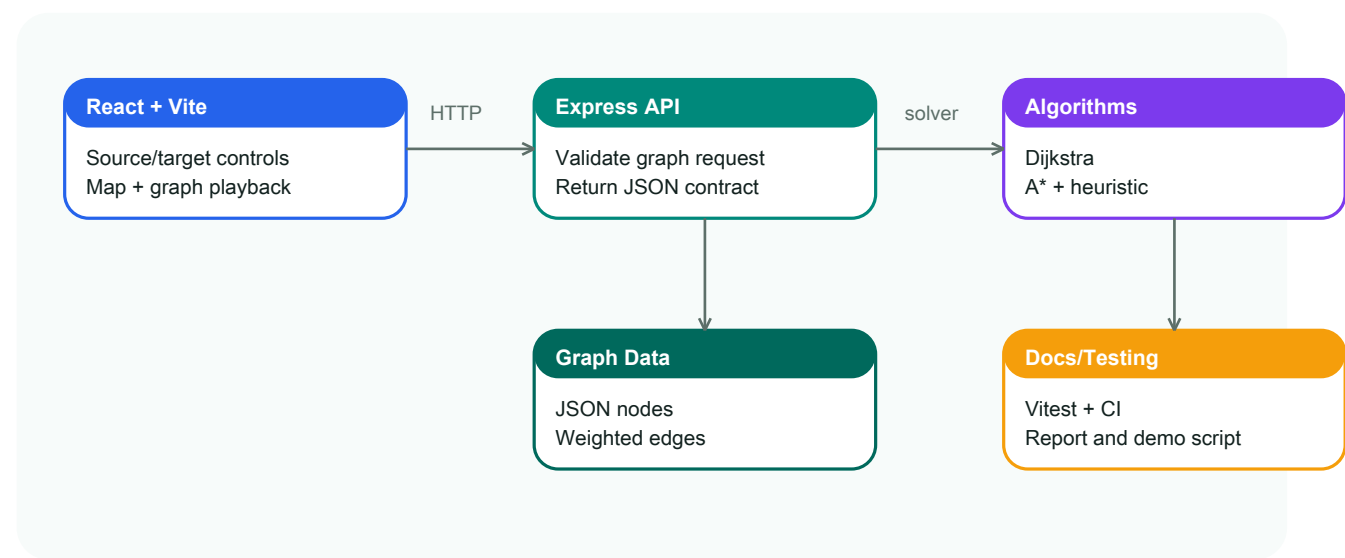
1. Tổng Quan Dự Án

Mục tiêu sản phẩm

RouteLab giúp trực quan hóa cách hai thuật toán shortest path tìm đường từ nguồn đến đích trên một graph bản đồ đơn giản. Đây là mô phỏng học thuật, không phải bản sao đầy đủ của Google Maps.

- Input chính là graph gồm nodes, edges, trọng số cạnh, source và target.
- Dijkstra bảo đảm shortest path với trọng số không âm bằng relaxation và priority queue.
- A* dùng thêm heuristic theo tọa độ để ưu tiên node có vẻ gần đích hơn.
- Frontend hiện dùng mock/precomputed result để demo UI; backend đã có contract và skeleton solver.

Kiến trúc tổng quan



Hình 1. Luồng dữ liệu từ UI sang API, solver, graph data và tài liệu kiểm thử.

2. Mô Hình Dữ Liệu Và API

Dataset demo dùng graph có trọng số. Mỗi cạnh biểu diễn một đoạn đường có thể đi giữa hai địa điểm; trọng số là chi phí mô phỏng như khoảng cách hoặc thời gian.

ID	Node	Lat	Lng	Vai trò
0	Central Post Office	10.7798	106.6990	Trung gian
1	Ben Thanh Market	10.7725	106.6980	Source
2	Independence Palace	10.7770	106.6953	Trung gian
3	Turtle Lake	10.7828	106.6955	Trung gian
4	Tan Dinh Church	10.7890	106.6908	Trung gian
5	War Remnants Museum	10.7794	106.6920	Trung gian
6	Saigon Zoo	10.7866	106.7057	Target

API contract

Endpoint	Vai trò	Kết quả chính
GET /api/datasets	Liệt kê graph demo	id, name, nodeCount, edgeCount
GET /api/datasets/:id	Lấy full dataset	nodes, edges, directed, defaultSource, defaultTarget
POST /api/solve/dijkstra	Chạy Dijkstra	path, totalCost, runtimeMs, visitedOrder, relaxedEdges
POST /api/solve/a-star	Chạy A*	path, totalCost, runtimeMs, visitedOrder, relaxedEdges

3. Thuật Toán Dijkstra

Dijkstra duy trì khoảng cách tốt nhất tạm thời từ source đến từng node. Ở mỗi vòng, thuật toán lấy node có distance nhỏ nhất ra khỏi priority queue rồi relax tất cả cạnh đi ra từ node đó.

1 Pick min

Lấy node có distance nhỏ nhất

2 Relax

Cập nhật cạnh đi ra nếu tốt hơn

3 Queue

Đưa distance mới vào priority queue

4 Stop

Dừng khi lấy target ra khỏi queue

```
dist[source] = 0
priorityQueue.push(source, 0)

while queue is not empty:
    current = pop node with smallest dist
    if current == target: stop

    for each edge current -> neighbor:
        nextCost = dist[current] + edge.weight
        if nextCost < dist[neighbor]:
            dist[neighbor] = nextCost
            previous[neighbor] = current
            queue.push(neighbor, nextCost)

return path reconstructed from previous[target]
```

Tiêu chí	Phân tích
Điều kiện đúng	Tất cả edge weight không âm.
Cấu trúc dữ liệu	Priority queue/min-heap cho node có distance nhỏ nhất.
Độ phức tạp	$O((V + E) \log V)$ nếu dùng binary heap.
Ưu điểm	Chắc chắn tìm shortest path, dễ giải thích bằng relaxation.
Hạn chế	Có thể duyệt nhiều node không hướng về target vì không dùng thông tin vị trí đích.

4. Thuật Toán A*

A* mở rộng Dijkstra bằng cách cộng thêm heuristic $h(n)$, tức ước lượng chi phí từ node hiện tại đến target. Nhờ đó thuật toán ưu tiên các node có tổng $f(n)$ nhỏ hơn.

A* priority

$$f(n) = g(n) + h(n)$$

$g(n)$: chi phí từ source đến n

$h(n)$: ước lượng từ n đến target bằng tọa độ

Nếu heuristic không vượt quá chi phí thật còn lại, A* vẫn tìm được shortest path.

```
g[source] = 0
f[source] = heuristic(source, target)
priorityQueue.push(source, f[source])

while queue is not empty:
    current = pop node with smallest f
    if current == target: stop

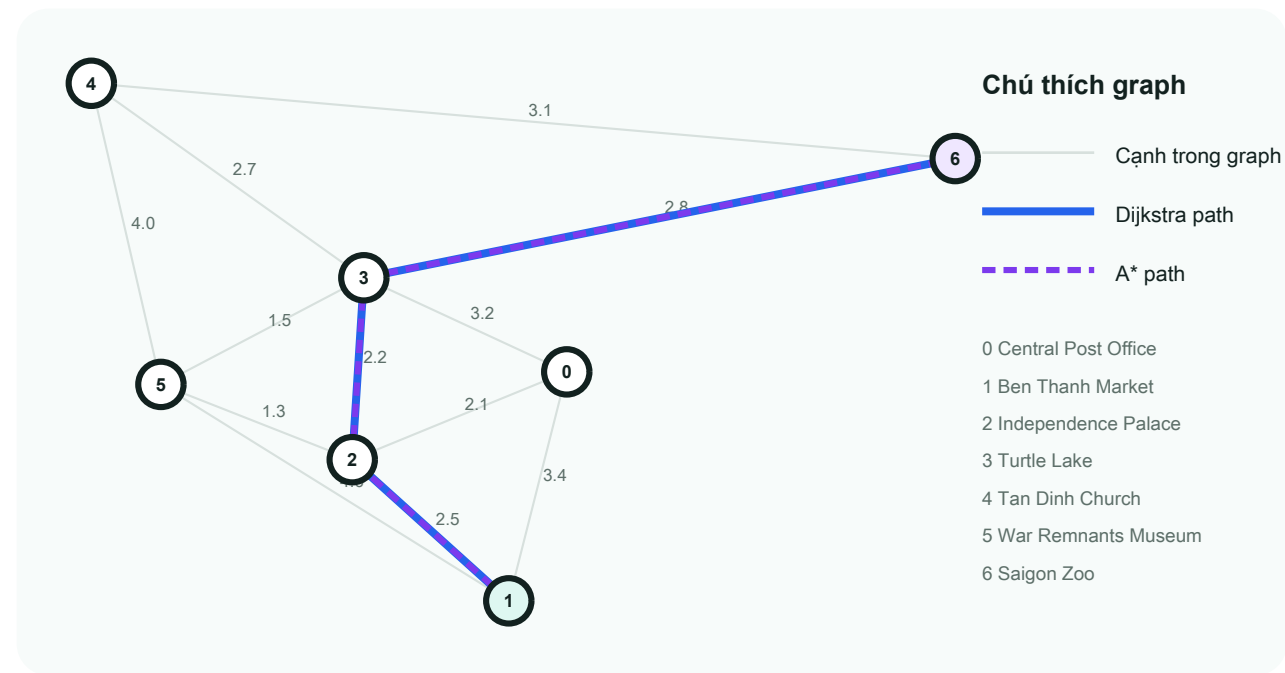
    for each edge current -> neighbor:
        tentativeG = g[current] + edge.weight
```

```
if tentativeG < g[neighbor]:  
    previous[neighbor] = current  
    g[neighbor] = tentativeG  
    f[neighbor] = g[neighbor] + heuristic(neighbor, target)  
    queue.push(neighbor, f[neighbor])
```

Tiêu chí	Phân tích
$g(n)$	Chi phí tốt nhất đã biết từ source đến node n.
$h(n)$	Ước lượng còn lại từ n đến target; demo dùng khoảng cách tọa độ đã scale bảo thủ.
$f(n)$	Độ ưu tiên trong queue: $f(n) = g(n) + h(n)$.
Tính tối ưu	Được giữ nếu heuristic admissible, tức không ước lượng vượt chi phí thật.
Kỳ vọng	Thường duyệt ít node hơn Dijkstra khi heuristic định hướng tốt.

5. Kết Quả Thực Nghiệm

Demo chạy từ source 1 (Ben Thanh Market) đến target 6 (Saigon Zoo). Runtime chỉ mang tính tham khảo vì phụ thuộc máy chạy và kích thước input.



Hình 2. Graph demo tại TP.HCM: cạnh xám là graph, xanh là Dijkstra path, tím nét đứt là A* path.

Thuật toán	Path	Total cost	Visited nodes	Runtime tham khảo
Dijkstra	1 → 2 → 3 → 6	7.5	1 → 2 → 0 → 5 → 3 → 4 → 6	0.023 ms
A*	1 → 2 → 3 → 6	7.5	1 → 2 → 0 → 5 → 3 → 6	0.026 ms

Kết luận demo

Cả hai thuật toán tìm path 1 → 2 → 3 → 6 với tổng chi phí 7.5. Dijkstra là baseline chắc chắn; A* cho cách giải thích trực quan hơn vì có heuristic hướng về đích.

Trace relaxation rút gọn

Bước	Sự kiện	Cạnh	Cost tích lũy
1	Relax edge	1 → 2	2.5
2	Relax edge	1 → 0	3.4
3	Relax edge	1 → 5	4.6
4	Relax edge	2 → 5	3.8
5	Relax edge	2 → 3	4.7
6	Relax edge	5 → 4	7.8
7	Relax edge	3 → 4	7.4
8	Relax edge	3 → 6	7.5
Done	Visited: 1 → 2 → 0 → 5 → 3 → 4 → 6	Path: 1 → 2 → 3 → 6	7.5

6. So Sánh Tổng Hợp

Tiêu chí	Dijkstra	A*
Mục tiêu	Tìm shortest path bằng distance hiện tại nhỏ nhất.	Tìm shortest path bằng distance + heuristic đến đích.
Bảo đảm tối ưu	Có nếu trọng số không âm.	Có nếu heuristic admissible/consistent.
Thông tin dừng thêm	Không dùng vị trí target để ưu tiên.	Dùng h(n), ví dụ khoảng cách tọa độ tới target.
Độ phức tạp	$O((V + E) \log V)$.	Worst-case tương tự Dijkstra, thực tế có thể duyệt ít hơn.
Phù hợp demo	Giải thích relaxation rõ ràng.	Giải thích mô phỏng bản đồ trực quan hơn.
Vai trò trong app	Baseline chuẩn để đối chiếu.	Phiên bản định hướng đích để so sánh số node đã duyệt.

Danh sách cạnh demo

Edge	From	To	Weight	Label
e0-1	0	1	3.4	Le Loi
e0-2	0	2	2.1	Dong Khoi
e0-3	0	3	3.2	Hai Ba Trung
e1-2	1	2	2.5	Le Thanh Ton
e1-5	1	5	4.6	Truong Dinh
e2-3	2	3	2.2	Pasteur
e2-5	2	5	1.3	Nam Ky Khoi Nghia
e3-4	3	4	2.7	Tran Cao Van
e3-5	3	5	1.5	Vo Van Tan
e3-6	3	6	2.8	Nguyen Binh Khiem
e4-5	4	5	4	Dien Bien Phu
e4-6	4	6	3.1	Nguyen Huu Canh

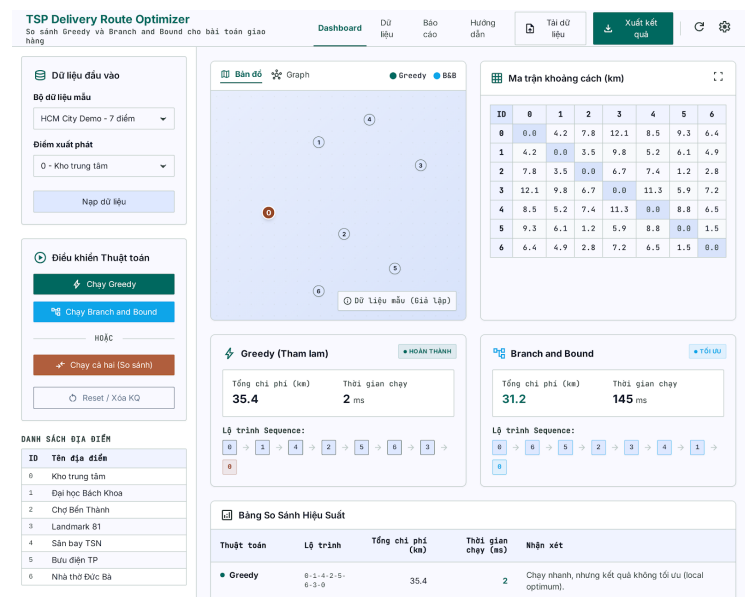
7. Kiểm Thử, CI/CD Và Hướng Phát Triển

- Backend có test cho health endpoint, dataset graph endpoint, validator graph và solve endpoints skeleton.
- Frontend có test cho validation, path cost, playback state, comparison table và mock solver contract.
- CI backend nên chặn deploy nếu typecheck/test/build lỗi hoặc AI review phát hiện lỗi nghiêm trọng.
- Vercel deploy frontend tại maps.hailamdev.space; backend sẽ triển khai riêng sau khi solver thật hoàn thiện.

Hướng phát triển sau migration

Hạng mục	Đề xuất cải thiện
Backend solver	Implement Dijkstra và A* thật, trả thêm visitedOrder/relaxedEdges để frontend replay đúng quá trình.
Visualization	Thêm lớp đường phố giả lập, animated vehicle, frontier heatmap, bảng priority queue theo từng step.
Data	Tạo dataset có nhiều ngã rẽ để A* duyệt ít node hơn Dijkstra rõ ràng.
Demo	Chuẩn bị kịch bản 3 phút: chọn nguồn/đích, chạy Dijkstra, bật A*, so sánh visited nodes và giải thích heuristic.
Report/slide	Đưa graph map, trace relaxation và bảng so sánh vào slide cuối kỳ.

Giao diện demo



Hình 3. Dashboard dùng để chọn dataset, source, target, chạy thuật toán và xem playback.